Backend Technical Audit Report

Cynbit Technologies - Public Website Review

Assessment: Backend Developer Intern - Analyze Cynbit from a Backend Perspective
Prepared by: Harsh Kumar
Audit date: 31 October 2026
Website: https://www.cynbit.in/
Testing approach: Passive, black-box review of public pages and normal user interactions

---

1. Executive Summary

This report evaluates publicly observable aspects of Cynbit Technologies' website from a backend engineering perspective. The review focuses on the public Contact inquiry workflow, browser-visible request behaviour, response metadata, data-handling considerations, reliability, security controls and performance measurement.

The most useful interaction observed was a normal submission through the public Contact form. The site displayed an **"INQUIRY RECEIVED"** confirmation and a confirmation email was received. In Chrome DevTools, a Fetch/XHR row named contact was observed with HTTP **200**, an approximately **2.58-second** browser duration and approximately **2.6 kB** transferred. These facts establish a successful user-visible workflow and a correlated network request. However, the captured screenshots do **not** show that request's exact URL, HTTP method, request payload or response body. Those details are therefore left unverified rather than guessed.

A separate blog navigation request showed GET 200, Content-Type: text/x-component, Server: Vercel, X-Matched-Path: /blog/[slug].rsc, X-Vercel-Cache: MISS, and an _rsc query parameter. This is consistent with a Next.js React Server Components navigation/data response served through Vercel. It is an architectural inference from response metadata, not proof of the complete application or data-layer architecture.

Analytics collection requests returning HTTP 204 were also observed. A 204 response is valid for a collection endpoint that returns no body and is not itself a failure.

No confirmed security vulnerability was established by the evidence available. Server-side validation, rate limiting, authentication boundaries for private systems, database persistence, email provider architecture, error paths and internal authorization could not be verified from the public browser session. The report therefore prioritizes practical improvement opportunities rather than making unsupported vulnerability claims.

Highest-value recommendations

1. Enforce server-side schema validation, field-length limits and request-size limits on public inquiry submissions.
2. Apply endpoint-specific abuse protection and rate limiting, backed by a shared store if the application runs on multiple instances.
3. Persist the inquiry before acknowledging successful receipt; use a transactional outbox or durable queue for email notifications.
4. Return consistent, minimal API responses and keep stack traces and personal data out of client responses and logs.
5. Measure server-side latency and report p50/p95/p99, separating database work from email-provider time.

6. Review cache policy by route: cache public static content where appropriate, but do not cache personalized or inquiry-related data.
7. Protect internal lead-management functionality with authentication, authorization and least privilege; the public form itself need not require login.

2. Scope, Rules and Methodology

2.1 In scope

- Public homepage: https://www.cynbit.in/
- Public Services page: https://www.cynbit.in/services
- Public Contact page: https://www.cynbit.in/contact
- Public Careers page: https://www.cynbit.in/careers
- Browser Network panel observations during normal page navigation and one normal Contact form submission
- Publicly visible request names, response status, timing, transferred-size and selected response headers
- Defensive design recommendations and illustrative backend code

2.2 Out of scope

- Source-code review or source-map extraction
- Private or undocumented API enumeration
- Authentication bypass or authorization testing
- SQL injection, XSS, CSRF exploitation or payload fuzzing
- Accessing other users' records or private data
- Automated vulnerability scanning, load testing, stress testing or repeated form submissions
- Direct database, cloud-console, server or mail-provider access

Only public pages and ordinary user interactions were used. No attempt was made to exploit, disrupt, overload or gain unauthorized access to Cynbit systems.

2.3 Method

1. Review public website content and identify user-facing workflows that involve data submission.
2. Open Chrome DevTools Network and filter to Fetch/XHR.
3. Observe ordinary navigation requests and response metadata.
4. Submit the Contact form once with candidate-provided test information.
5. Correlate the visible confirmation, the network request row and the received confirmation email.
6. Record only values actually visible in screenshots. Do not infer missing URL, method, payload or response values.
7. Develop defensive recommendations and a separate local reference implementation.

2.4 Evidence confidence labels

- **Observed:** directly visible in a browser screenshot or directly experienced during the normal interaction.
- **Consistent with:** technical interpretation strongly indicated by observed headers or request patterns, but not a source-code confirmation.
- **Not verified:** the available evidence does not establish whether a control exists or how it is implemented.
- **Recommendation:** a defensive design improvement; not an assertion that Cynbit currently has a defect.

3. Public Website Context

The public homepage describes Cynbit as working on AI automation, autonomous AI agents, custom software development and scalable business platforms [1]. The Services page publishes a broad technology capability list that includes TypeScript, Python, Go, React, Next.js, NestJS, FastAPI, PostgreSQL, MongoDB, Redis, AWS,

Azure, Docker and Kubernetes [3]. This list describes publicly advertised capabilities; it does not prove that all of these technologies are used by the website reviewed in this audit.

The Contact page presents a project inquiry path and describes a discovery process in which a visitor provides project requirements [2]. The inspected form showed fields for full name, email address, optional company, optional industry and a free-text message. The Careers page describes a remote-first model and lists a Backend Developer Intern role [4].

The public website content is useful for understanding the intended business workflows, but it is not a substitute for access to internal source code or architecture documentation.

---

4. Observed Architecture and Request Behaviour

4.1 Separate blog route request - Next.js/RSC indicators

A request captured while viewing a blog route showed the following values:

Attribute	Observed value
Request path	/blog/ai-agents-business-operations?_rsc=...
Method	GET
Status	200 OK
Content-Type	text/x-component
Server	Vercel
X-Matched-Path	/blog/[slug].rsc
X-Vercel-Cache	MISS
Query parameter	_rsc present
Cache-Control	private, no-cache, no-store, max-age=0, must-revalidate

Interpretation: The _rsc parameter, text/x-component content type and .rsc matched path are consistent with Next.js React Server Components navigation/data transport. The Server: Vercel header indicates that the observed response was served through Vercel infrastructure. This does not establish the backend database, private API framework or internal service topology.

Cache interpretation: The selected response was marked non-cacheable by its Cache-Control header and the Vercel cache status was MISS. This may be intentional for request-specific or dynamic content. A single cache miss does not establish a performance defect.

4.2 Contact form submission - correlated request

Attribute	Observed value
Network row name	contact
Type	fetch
Status	200
Browser duration	Approximately 2.58 s
Transferred size	Approximately 2.6 kB
User-visible result	"INQUIRY RECEIVED" confirmation
Email result	Confirmation email received

Interpretation: The contact-related Fetch request was observed in the same interaction that led to the successful confirmation screen and email. This supports correlation with the inquiry workflow.

Important evidence gap: The request was not selected for inspection in the final screenshot set. The exact Request URL, HTTP method, payload, response headers and response body are not available. The report

therefore does not assert that the route is POST /api/contact, does not reproduce a payload, and does not claim that a database insert was independently verified.

The list also contained a separate contact?utm_source=...&_rsc=... navigation request. It should not be confused with the row named contact.

4.3 Analytics collection

Requests matching an Analytics collect endpoint were visible with status 204. A 204 response means the request succeeded without a response body. This status is normal for some telemetry collection endpoints. The evidence does not reveal the complete analytics configuration or prove which fields were transmitted.

Recommendation: Ensure form messages, email addresses and other personal or confidential details are not included in analytics event parameters or URLs. Use consent and retention settings appropriate to the applicable privacy requirements.

4.4 Request inventory and limitations

Request group	What was captured	What remains unknown
Contact submission	Row contact, Fetch, 200, ~2.58 s, ~2.6 kB	Exact URL, method, payload, response schema, database persistence
Contact RSC navigation	contact?..._rsc... row visible	Internal server action/navigation implementation
Blog RSC navigation	GET 200, text/x-component, Vercel, .rsc matched path	Full Next.js version and backend services
Analytics	collect requests, 204	Exact event payload/configuration and consent behaviour

5. Security and Data Handling Findings

No confirmed exploitable security issue was identified. Findings below distinguish observable facts from controls that could not be verified.

S-01 - Server-side input validation

Category: Input validation
Priority: High defensive priority
Status: Not verified

The Contact form visibly accepts a name, email address, optional company, optional industry and a free-text message. Browser-side required-field checks, if present, would not prove that the server validates the same fields.

Risk if absent: Malformed, unexpected or excessively long input may reach application logic, database operations or notification services. Unexpected properties can also cause inconsistent data handling.

Recommendation: Apply an allow-listed schema at the server boundary. Trim and normalize values, enforce minimum/maximum lengths, validate email syntax, reject unknown keys, constrain content types and keep the request body small. Use parameterized SQL rather than string concatenation.

Evidence needed to verify: Authorized inspection of the actual request and response, followed by approved validation tests against a test environment. No malformed payload testing was performed against Cynbit.

S-02 - Public form abuse and rate limiting

Category: Abuse prevention / availability
Priority: High defensive priority
Status: Not verified

The inquiry form is publicly accessible. The screenshots do not establish whether rate limiting, bot detection, duplicate suppression or edge-level abuse protection is configured.

Risk if absent: Automated submissions may create unwanted lead records, trigger unnecessary email or CRM work, and consume backend resources.

Recommendation: Use per-IP and per-endpoint rate limits, a shared rate-limit store for horizontally scaled deployments, request-size constraints, monitoring and proportionate bot controls. Consider idempotency for client retries and avoid blocking legitimate users behind shared corporate/NAT IPs with overly strict limits.

S-03 - Personal data minimization, retention and logging

Category: Data protection
Priority: High defensive priority
Status: Form fields observed; storage and logging controls not verified

The public form displays fields for full name, email, optional company, optional industry and message. These fields may contain personal or commercially sensitive details.

Risk if poorly handled: Excessive retention, broad staff access, verbose request logging or sending form fields to analytics can expose information beyond the intended inquiry workflow.

Recommendation: Collect only what is needed for lead follow-up, restrict access by role, define a retention/deletion schedule, avoid placing personal data in URLs or analytics events, redact request bodies from logs, and ensure transport/storage encryption is configured by the hosting and database layers.

S-04 - Authentication and authorization boundary

Category: Authentication / authorization
Priority: Medium architectural priority
Status: Not applicable to the public inquiry workflow; internal access not tested

A public inquiry form does not ordinarily need a user account. Requiring login may create unnecessary friction. However, any internal dashboard used to review or export submissions should be protected.

Recommendation: Require authentication for internal lead-management interfaces, enforce authorization on every API request, use least-privilege roles, MFA where available, secure session cookies and auditable administrative actions. Do not rely on hiding UI routes as an authorization control.

S-05 - Transport and security headers

Category: Transport / browser security
Priority: Medium
Status: Partial evidence

The separate RSC response was captured over HTTPS and showed Strict-Transport-Security: max-age=63072000. This is a positive transport-security indicator for that response. The screenshots do not provide a complete header review across all application routes.

Recommendation: Maintain HTTPS/HSTS, review CSP and other response headers across relevant routes, and validate policies against actual scripts, fonts and integrations before enforcement. Use a security-header baseline such as Helmet in an Express service where applicable.

S-06 - Error handling and information disclosure

Category: Error handling
Priority: Medium
Status: Not verified

Only a successful inquiry flow was observed. No server error or validation failure response was captured.

Recommendation: Use a consistent error response format with stable error codes and a request ID. Return generic messages for unexpected server errors, keep stack traces and provider/database details in access-controlled logs, and never log cookies, tokens or complete form messages by default.

---

6. Performance and Reliability Findings

P-01 - Contact request duration

Category: API performance
Priority: Medium investigation priority
Status: One browser sample observed

The Fetch/XHR row named contact showed about 2.58 seconds and about 2.6 kB transferred. This is a browser-observed duration, not a server execution-time measurement. It may include network time, server processing, connection reuse, platform scheduling and other browser/network factors. One sample cannot establish a systemic performance issue.

Recommendation: Add server-side timing and distributed tracing. Report p50/p95/p99 latency and separate validation, database commit, queue publish and email delivery durations. A request should not wait for a third-party email provider if the business process can safely send notifications asynchronously after durable persistence.

P-02 - Reliable email notification

Category: Reliability / asynchronous processing
Priority: High design opportunity
Status: Email receipt observed; internal delivery design not verified

A confirmation email arrived after the inquiry confirmation. This proves the candidate received an email, but does not reveal whether delivery was synchronous, queued, retried or sent by a third-party provider.

Recommendation: Persist the inquiry and a notification outbox record in one database transaction. A background worker should claim pending events, send notifications, record attempt counts, retry transient failures with exponential backoff and expose permanently failed events for operational review. This avoids losing an inquiry when email delivery is temporarily unavailable.

P-03 - Caching policy by route

Category: CDN/cache efficiency and data isolation
Priority: Medium investigation priority
Status: One RSC response observed

The selected blog RSC response included Cache-Control: private, no-cache, no-store, max-age=0, must-revalidate and X-Vercel-Cache: MISS. This may be appropriate for a dynamic or request-specific response. It is not enough to determine whether public pages are optimally cached.

Recommendation: Classify routes as public-static, public-dynamic or personalized/private. Use CDN caching and revalidation for suitable public content. Keep inquiry submissions, private user data and personalized responses non-cacheable. Validate cache keys and Vary behaviour to prevent cross-user response reuse.

P-04 - Payload size and database efficiency

Category: Resource efficiency
Priority: Medium
Status: Request payload not captured

The contact row showed about 2.6 kB transferred, but its request body was not inspected. The observation does not establish the submitted payload size or database query complexity.

Recommendation: Enforce request-size and per-field limits. Use database constraints, parameterized queries and indexes aligned with actual query patterns. If future endpoints return large inquiry lists, apply pagination and avoid unbounded queries.

Performance measurement plan

For any further authorized verification, capture a small representative sample of normal user requests and correlate browser timings with server/APM metrics. Track:

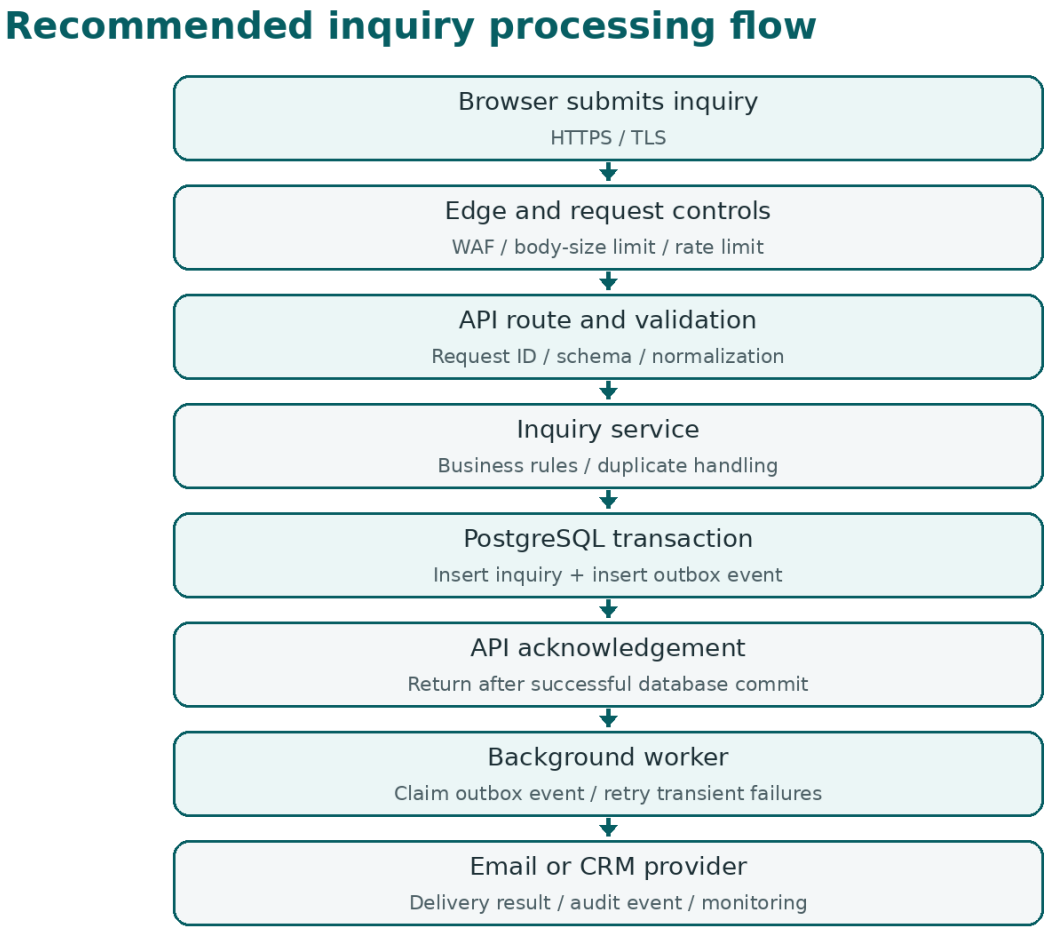
- p50, p95 and p99 request duration
- 4xx and 5xx rates by route
- database query duration and connection-pool saturation
- queue depth and oldest pending notification age
- email delivery success/failure and retry count
- CDN cache hit ratio for public static content

No automated performance testing or load testing was performed.

---

7. Recommended Backend Design

7.1 Suggested flow for inquiry processing



7.2 Suggested API contract (illustrative only)

A documented route such as POST /api/v1/inquiries could accept:

- fullName (required, bounded length)
- emailAddress (required, valid email format)
- company (optional, bounded length)
- industry (optional, bounded length)
- message (required, bounded length)

The exact field names and endpoint used by Cynbit were not captured. This contract is a proposed design only.

If a record is durably created before the response, 201 Created with a minimal inquiry ID and timestamp is semantically clear. If the system only accepts work for later processing and cannot yet confirm durable creation, 202 Accepted may be more appropriate. A 200 OK response is not automatically a defect; status semantics should match the actual operation.

7.3 Data storage and notification pattern

Use PostgreSQL constraints and parameterized queries. Store the inquiry and an outbox event in one transaction. A worker can process the event after commit. Avoid returning or logging the full submitted message. Define retention and deletion rules for lead records.

7.4 Authentication model

Keep the public inquiry entry point simple and unauthenticated if the use case requires public access, but apply spam/abuse controls. Require authentication and authorization for internal dashboards, exports and administrative APIs. Authorization must be enforced on the server for every record and action.

---

8. Reference Implementation Included

The accompanying repository includes a local Express 5 + PostgreSQL reference implementation demonstrating:

- Helmet security headers
- Exact-origin CORS configuration for cross-origin use
- JSON request body size limit
- Global and contact-specific rate limiting
- Zod schema validation and rejection of unknown fields
- Parameterized PostgreSQL queries
- Transactional inquiry + outbox persistence
- Minimal success response with request ID
- Centralized error handling that avoids exposing internal details
- Health endpoint and environment-based configuration

The reference code is **not Cynbit source code**, is not connected to Cynbit systems and must not be deployed against their infrastructure. It is a demonstration of recommended engineering controls. The sample uses an in-memory rate limiter and intentionally does not include a real email worker; those are stated limitations in the repository README.

8.1 Server-side validation example

```
const { z } = require('zod');

const inquirySchema = z.object({
  fullName: z.string().trim().min(2).max(80),
  emailAddress: z.string().trim().email().max(254),
  company: z.string().trim().max(120).optional(),
  industry: z.string().trim().max(100).optional(),
  message: z.string().trim().min(10).max(2000)
}).strict();

const parsed = inquirySchema.safeParse(req.body);
if (!parsed.success) {
  return res.status(400).json({
    success: false,
    error: { code: 'VALIDATION_ERROR', message: 'Please check the submitted fields.' }
  });
}
```

This is an illustrative schema. The actual live field names were not captured in the browser payload, so the sample contract must be adapted after authorized inspection.

8.2 Parameterized persistence and transactional outbox example

```
await client.query('BEGIN');
const result = await client.query(
  `INSERT INTO inquiries (full_name, email_address, message)
   VALUES ($1, $2, $3) RETURNING id`,
  [fullName, emailAddress, message]
);
```

```
await client.query(
  `INSERT INTO outbox_events (event_type, payload)
   VALUES ($1, $2::jsonb)`,
  ['inquiry.received', JSON.stringify({ inquiryId: result.rows[0].id })]
);
await client.query('COMMIT');
return res.status(201).json({
  success: true,
  data: { inquiryId: result.rows[0].id }
});
```

The two database writes commit together. A separate worker can send email after commit, retry transient provider failures and alert on exhausted retries.

8.3 Rate limiting and request-size controls

```
app.use(express.json({ limit: '10kb' }));
app.use('/api/contact', rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 5,
  standardHeaders: 'draft-7',
  legacyHeaders: false
}));
```

The example uses an in-memory limiter for a local demonstration. Use a shared Redis-compatible store behind multiple application instances, and tune limits to avoid harming legitimate users.

8.4 Centralized error handling

```
function errorHandler(err, req, res, next) {
  if (res.headersSent) return next(err);
  const status = err.statusCode || err.status || 500;
  res.status(status).json({
    success: false,
    error: {
      code: status >= 500 ? 'INTERNAL_ERROR' : 'REQUEST_ERROR',
      message: status >= 500 ? 'An unexpected error occurred.' : err.message
    },
    requestId: req.requestId
  });
}
```

Keep detailed diagnostics in protected logs. Do not return stack traces, database errors, secrets or full inquiry bodies to the browser.

---

9. Prioritized Recommendations

Priority	Recommendation	Benefit and verification
High	Server-side field validation and request-size limits	Protects downstream processing and data quality. Verify the API schema and approved test cases in a test environment.
High	Abuse/rate controls on public inquiry endpoint	Reduces spam and unnecessary processing. Review edge/application configuration and monitoring.
High	Durable inquiry persistence + outbox notification	Prevents email-provider failures from losing inquiries.

Priority	Recommendation	Benefit and verification
		Review transaction and worker design.
High	PII-safe logging, access control and retention	Reduces data exposure. Review logging policy, roles and retention settings.
Medium	Consistent errors and request correlation IDs	Improves support and incident diagnosis. Inspect validation and failure responses in a test environment.
Medium	Server-side latency metrics	Separates network delay from application bottlenecks. Compare browser timing with APM traces.
Medium	Route-based cache policy	Improves public-page performance without leaking private data. Review cache headers and cache-key rules by route.
Medium	Authentication and authorization for internal tools	Protects lead records and administrative actions. Conduct an authorized access-control review.

10. Conclusion

The public Contact workflow was successfully exercised once: the site displayed an inquiry-received confirmation and a confirmation email arrived. The browser showed a correlated contact Fetch request returning HTTP 200 in approximately 2.58 seconds. A separate blog request exposed Next.js/RSC-style response metadata served through Vercel, and analytics collection requests returned HTTP 204.

The available evidence is sufficient to describe these observable behaviours, but it is not sufficient to verify the contact endpoint's exact API contract, server-side validation, data persistence, notification architecture, authentication, authorization or error handling. No confirmed vulnerability is reported.

The strongest engineering opportunity is to ensure that public inquiry intake is validated, abuse-resistant and durably persisted, while email notifications are handled asynchronously and observably. The included Express/PostgreSQL reference code demonstrates one way to implement these controls.

This report reflects a limited public black-box assessment, not a source-code, infrastructure or penetration test.

Appendix A - Evidence Register

ID	Evidence and supported statement	Limitation
E-01	Contact page + Network list: row contact, Fetch, 200, ~2.58 s, ~2.6 kB	URL, method, payload and response body not captured
E-02	Inquiry confirmation page: site displayed "INQUIRY RECEIVED"	UI confirmation alone does not prove database durability

ID	Evidence and supported statement	Limitation
E-03	Confirmation email: email arrived after submission	Internal provider and queue design unknown
E-04	Blog RSC Headers: GET 200, text/x-component, Vercel, .rsc matched path, cache MISS	Separate navigation request, not contact submission
E-05	Blog RSC Payload/Response: _rsc parameter and serialized RSC response	Not the contact form payload
E-06	Analytics rows: collection requests returned 204	Exact analytics payload/configuration unknown

Only redacted screenshots should be included in a public submission. The screenshots shared during the working session included visible personal details; those originals should not be committed without redaction.

Appendix B - Additional evidence that would strengthen the audit

If time and authorized access permit, capture the contact request's Headers, Payload and Response tabs. Redact email addresses, message content, cookies and tokens. Record the actual URL and method; do not assume the endpoint is REST or that it uses a JSON payload. A separate approved review of validation failures, rate-limit configuration, internal authentication, retention and notification retries would be needed to verify those controls.

References

[1] Cynbit Technologies, Homepage, https://www.cynbit.in/ (accessed 31 October 2026).
[2] Cynbit Technologies, Contact page, https://www.cynbit.in/contact (accessed 31 October 2026).
[3] Cynbit Technologies, Services page, https://www.cynbit.in/services (accessed 31 October 2026).
[4] Cynbit Technologies, Careers page, https://www.cynbit.in/careers (accessed 31 October 2026).